🎯 Flask Routing Challenge: Movie Finder API

Scenario

You are building a simple **Movie Finder application**.

The application stores movie information in a Python dictionary. Users should be able to search for movies using different information from the URL.

Given Data

```
movies = {
    "M101": {
        "title": "RRR",
        "director": "S. S. Rajamouli",
        "language": "Telugu",
        "genre": "Action",
        "rating": 8,
        "year": 2022
    },
    "M102": {
        "title": "Pushpa",
        "director": "Sukumar",
        "language": "Telugu",
        "genre": "Action",
        "rating": 7,
        "year": 2021
    },
    "M103": {
        "title": "Jersey",
        "director": "Gowtam Tinnanuri",
        "language": "Telugu",
        "genre": "Drama",
        "rating": 9,
        "year": 2019
    },
    "M104": {
        "title": "Bahubali",
        "director": "S. S. Rajamouli",
        "language": "Telugu",
        "genre": "Action",
        "rating": 9,
```

```
        "year": 2015
    },
    "M105": {
        "title": "KGF",
        "director": "Prashanth Neel",
        "language": "Kannada",
        "genre": "Action",
        "rating": 8,
        "year": 2018
    },
    "M106": {
        "title": "3 Idiots",
        "director": "Rajkumar Hirani",
        "language": "Hindi",
        "genre": "Comedy",
        "rating": 9,
        "year": 2009
    }
}
```

🎬 Your Mission

Build a Flask application where a user can find movies **just by changing the URL**.

You are **not given the route implementation**.

You need to decide:

> Which routes should be static?
> Which routes should be dynamic?
> Where should `<string>` be used?
> Where should `<int>` be used?
> What should happen when data doesn't exist?

🚪 Level 1 – Movie Theatre Entrance

Create a route that displays:

```
Welcome to Movie Finder
```

The URL should be:

```
/
```

Concept: Static routing.

🎥 Level 2 – Show All Movies

Create a route that displays all available movies.

URL:

```
/movies
```

Concept: Static routing + dictionary response.

🔎 Level 3 – Find Your Movie

A user knows the movie ID.

For example:

```
/movies/id=M103
```

Your application should return the details of `Jersey`.

If the user enters:

```
/movies/id=M999
```

display:

```
Movie not found
```

Concept: Dynamic routing.

🎭 Level 4 – Director Search

A user wants to find all movies directed by:

```
S. S. Rajamouli
```

The URL should follow this pattern:

```
/movies/director=<director_name>
```

For example:

```
/movies/director=S. S. Rajamouli
```

Expected movies:

```
RRR
Bahubali
```

Also test:

```
/movies/director=s. s. rajamouli
```

It should work.

Requirement: Search should be case-insensitive.

⭐ Level 5 – Rating Filter

A user wants movies with a particular rating.

For example:

```
/movies/rating=9
```

Expected:

```
Jersey
Bahubali
3 Idiots
```

Important

The rating must be received as an **integer**.

So think about which Flask converter you need:

```
<int:_____>
```

Now test:

```
/movies/rating=abc
```

Observe what Flask does.

🌎 Level 6 – Language Finder

Create a route that allows users to search movies by language.

Example:

```
/movies/language=Telugu
```

Expected:

```
RRR
Pushpa
Jersey
Bahubali
```

Also test:

```
/movies/language=telugu
```

It should work.

Test:

```
/movies/language=English
```

Return:

```
No movies found
```

🎞️ Level 7 – Genre Finder

Create a route to search by genre.

Example:

```
/movies/genre=Action
```

Expected:

```
RRR
Pushpa
Bahubali
KGF
```

Test:

```
/movies/genre=Comedy
```

Expected:

```
3 Idiots
```

🕐 Level 8 – Year Search

A user wants to watch movies released in a particular year.

Create a route such as:

```
/movies/year=2022
```

Expected:

```
RRR
```

Use an **integer route converter**.

Also test:

```
/movies/year=2025
```

Expected:

```
No movies found for this year
```

🔥 Level 9 – Double Filter

Now the challenge becomes interesting.

A user wants:

Telugu Action movies

Your URL should contain **both language and genre**.

For example:

```
/movies/language=Telugu/genre=Action
```

Expected:

```
RRR
Pushpa
Bahubali
```

Now try:

```
/movies/language=Kannada/genre=Action
```

Expected:

```
KGF
```

And:

```
/movies/language=Hindi/genre=Action
```

Expected:

```
No movies found
```

Concept being tested

Students must understand:

```
Dynamic parameter 1
        +
Dynamic parameter 2
        ↓
Check both conditions
```

🧪 Level 10 – Breaking the Application

Now deliberately give the application **wrong inputs**.

Students must test all of these:

```
/movies/id=M999
```

```
/movies/director=Unknown
```

```
/movies/language=English
```

```
/movies/genre=Horror
```

```
/movies/rating=1
```

```
/movies/year=2025
```

```
/movies/rating=abc
```

```
/movies/year=hello
```

For every case, students should understand **why the result occurs**

Day- 4

🎬 Day 4 – Movie Finder Web Application Extension

🎯 Objective

Convert the URL-based Movie Finder API into a simple **HTML-based Movie Finder application**.

Students should now learn how:

```
HTML Form
    ↓
GET / POST request
    ↓
Flask route
    ↓
request.form
    ↓
Search movies
    ↓
render_template()
```

and:

```
Form submission
      ↓
POST route
      ↓
Process data
      ↓
redirect()
      ↓
GET route
```

🏠 Level 1 – Movie Home Page

Create a home page using:

`/`

Instead of returning plain text, create:

```
templates/
    home.html
```

The page should display:

```
🎬 Movie Finder

Welcome to Movie Finder

[View All Movies]
[Search Movie]
[Search by Director]
[Search by Language]
```

Concepts

- `render_template()`
- HTML links
- `url_for()`

📽️ Level 2 – Display Movies Using Template

Modify:

`/movies`

Instead of returning the dictionary directly, display movies in an HTML table.

Example:

M101	RRR	S. S. Rajamouli	Telugu	Action	8	2022

Create:

```
templates/
    movies.html
```

Use Jinja:

```
{% for movie_id, movie in movies.items() %}
```

Concepts

- `render_template()`
- Jinja `for` loop
- Dictionary data → HTML

🔎 Level 3 – Search Movie Using Form

Create:

```
/search
```

Display a form:

```
Enter Movie ID:

[_____________]

[Search Movie]
```

The form should use:

```
<form method="POST">
```

When the user enters:

```
M103
```

the Flask application should display:

```
Movie Found!

Title: Jersey
Director: Gowtam Tinnanuri
Language: Telugu
Genre: Drama
Rating: 9
Year: 2019
```

Flask concepts being tested

`request.method`
`request.form`
`render_template()`

🔄 Level 4 – GET vs POST Challenge

Students must create **two different behaviours** for `/search`.

GET

When the user visits:

`/search`

display the search form.

POST

When the user submits the form:

`/search`

receive the movie ID using:

`request.form`

and search the dictionary.

Expected flow

```
GET /search
     ↓
Display search form
     ↓
User enters M103
     ↓
POST /search
     ↓
request.form
     ↓
Search movies
     ↓
Display result
```

🎭 Level 5 – Director Search Form

Create:

```
/director-search
```

Form:

```
Enter Director Name:

[_____________________________]

[Search]
```

If the user enters:

```
S. S. Rajamouli
```

display:

```
Movies Directed By S. S. Rajamouli

RRR
Bahubali
```

The search must be **case-insensitive**.

So this should also work:

```
s. s. rajamouli
```

Concept

Students must use:

```
request.form
```

and compare using something like:

```
movie["director"].lower()
```

🌎 Level 6 – Language Search Form

Create:

```
/language-search
```

Form:

```
Enter Language:

[_____________]

[Search Movies]
```

Input:

```
Telugu
```

Expected:

```
RRR
Pushpa
Jersey
Bahubali
```

Input:

```
English
```

Expected:

```
No movies found
```

Search should be case-insensitive.

⭐ Level 7 – Rating Search Form

Create:

```
/rating-search
```

Form:

```
Enter Rating:

[____]

[Search]
```

If the user enters:

```
9
```

display:

```
Jersey
Bahubali
3 Idiots
```

Challenge

The value from:

```
request.form
```

will initially be a **string**.

For example:

```
rating = request.form["rating"]
```

gives:

```
"9"
```

Students must convert it:

```
rating = int(rating)
```

Then compare it with the dictionary value.

➕ Level 8 – Add a New Movie

Now make the application more interesting.

Create:

```
/add-movie
```

Display a form:

```
Movie ID:       [_________]

Title:          [_________]

Director:       [_________]

Language:       [_________]

Genre:          [_________]

Rating:         [_________]

Year:           [_________]
```

```
[Add Movie]
```

When the form is submitted, add the movie to the dictionary.

For example:

```
M107
Devara
Koratala Siva
Telugu
Action
8
2024
```

After submission:

```
Movie added successfully!
```

Concepts

- `POST`
- `request.form`
- Dictionary insertion
- `redirect()`
- `url_for()`

🔄 Level 9 – Redirect Challenge

After successfully adding a movie, **do not directly render the movies page from the POST request**.

Instead:

```
POST /add-movie
       ↓
Add movie
       ↓
redirect()
       ↓
/movies
       ↓
GET /movies
       ↓
```

```
Display all movies
```

Students should implement:

```
return redirect(url_for("movies"))
```

🎯 Why?

This introduces students to the **POST → Redirect → GET** pattern.

It also prevents accidental duplicate form submissions when the user refreshes the page.

⚠️ Level 10 – Validation Challenge

Now deliberately enter incorrect data.

Students must handle:

Movie ID already exists

```
M101
```

Expected:

```
Movie ID already exists
```

Empty title

Expected:

```
Title cannot be empty
```

Invalid rating

```
abc
```

Expected:

```
Rating must be a number
```

Invalid year

```
hello
```

Expected:

```
Year must be a number
```

🚀 Level 11 – Delete Movie

Create:

```
/delete-movie
```

Create a form:

```
Enter Movie ID:

[_________]

[Delete Movie]
```

For:

```
M103
```

delete Jersey.

Expected:

```
Movie deleted successfully!
```

If:

```
M999
```

then:

```
Movie not found
```

Concepts

Students should use:

```
request.method == "POST"
```

```
request.form
```

```
del movies[movie_id]
```

and:

```
redirect(url_for("movies"))
```

🏆 Final Challenge – Movie Finder Dashboard

Finally, create:

```
/dashboard
```

The dashboard should contain buttons/links for:

```
🎬 Movie Finder

--------------------------------

[ View All Movies ]

[ Search Movie ]

[ Search by Director ]

[ Search by Language ]

[ Search by Genre ]
```

```
[ Search by Rating ]

[ Search by Year ]

[ Add Movie ]

[ Delete Movie ]

------------------------------
```

Every button should navigate to the appropriate Flask route.

📁 Suggested Project Structure

Students should finish Day 4 with:

```
movie_finder/
│
├── app.py
│
├── templates/
│   ├── home.html
│   ├── dashboard.html
│   ├── movies.html
│   ├── search.html
│   ├── director_search.html
│   ├── language_search.html
│   ├── genre_search.html
│   ├── rating_search.html
│   ├── year_search.html
│   ├── add_movie.html
│   └── delete_movie.html
│
└── static/
    └── style.css
```